# Pertemuan 4
# Debugging, Testing, Terminasi, dan Rekursi

## Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi
Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera

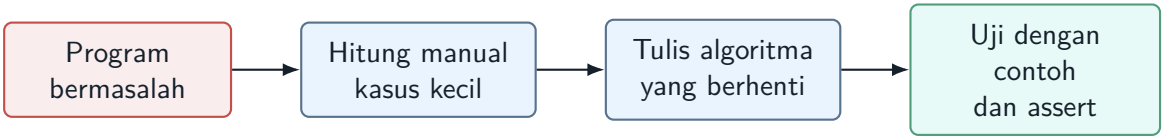
Semester Ganjil 2026/2027

### Agenda hari ini

- Lihat permasalahan pada kode.
- Hitung manual untuk kasus kecil.
- Tulis algoritma yang jelas berhenti.
- Jenis-jenis: error, exception, testing, terminasi.



# Dari masalah ke perbaikan




**Aturan kelas hari ini**

Sebelum melihat/membuat kode, coba prediksi hasil pseudocodenya. Baru uji coba.

# Masalah 1: rata-rata yang aneh

```
def rata(data):
    total = 0
    for x in data:
        total += x
        return total / len(data)

print(rata([2, 4, 6, 8]))
```

## Diskusi dulu

1. Secara matematis, rata-rata data itu berapa?
2. Menurut kode, baris mana yang dijalankan lebih dulu?
3. Apakah semua data sempat diproses?

# Masalah 1: rata-rata yang aneh

```
def rata(data):
    total = 0
    for x in data:
        total += x
        return total / len(data)

print(rata([2, 4, 6, 8]))
```

## Diskusi dulu

1. Secara matematis, rata-rata data itu berapa?
2. Menurut kode, baris mana yang dijalankan lebih dulu?
3. Apakah semua data sempat diproses?

## Hasil yang perlu disadari

Kode mengembalikan nilai terlalu cepat: hanya elemen pertama yang dipakai.

# Hitung manual, lalu perbaiki algoritma

## Perhitungan manual

Tutup dulu hasilnya. Hitung di kertas:
$[2, 4, 6, 8]$.

# Hitung manual, lalu perbaiki algoritma

## Perhitungan manual

$$\bar{x} = \frac{2+4+6+8}{4} = 5.$$

Algoritma harus menjumlahkan semua elemen dulu, baru membagi.

```
def rata(data):
    total = 0
    for x in data:
        total += x
    return total / len(data)
```

# Hitung manual, lalu perbaiki algoritma

## Perhitungan manual

$$\bar{x} = \frac{2+4+6+8}{4} = 5.$$

Algoritma harus menjumlahkan semua elemen dulu, baru membagi.

```python
def rata(data):
    total = 0
    for x in data:
        total += x
    return total / len(data)
```

```python
def rata_aman(data):
    if len(data) == 0:
        return None
    total = 0
    for x in data:
        total += x
    return total / len(data)
```

# Masalah 2: membagi dua nilai dari input

```
def rasio(a, b):
    a = float(a)
    b = float(b)
    return a / b
```

## Prediksi kasus uji

Apa yang terjadi untuk input berikut?

1. `('3.5','2')`
2. `('abc',2)`
3. `(3,0)`
4. `(None,4)`

# Masalah 2: membagi dua nilai dari input

```
def rasio(a, b):
    a = float(a)
    b = float(b)
    return a / b
```

## Prediksi kasus uji

Apa yang terjadi untuk input berikut?

1. `('3.5','2')`
2. `('abc',2)`
3. `(3,0)`
4. `(None,4)`

## Ide

Tidak semua kegagalan sama. Karena itu pesan kesalahan sebaiknya dipisah.

# Membuat fungsi rasio lebih terkendali

```
def rasio(a, b):
    try:
        a = float(a)
        b = float(b)
        return a / b
    except ValueError:
        return "input tidak dapat diubah menjadi angka"
    except TypeError:
        return "tipe data input tidak sesuai"
    except ZeroDivisionError:
        return "penyebut tidak boleh nol"
```

# Membuat fungsi rasio lebih terkendali

```
def rasio(a, b):
    try:
        a = float(a)
        b = float(b)
        return a / b
    except ValueError:
        return "input tidak dapat diubah menjadi angka"
    except TypeError:
        return "tipe data input tidak sesuai"
    except ZeroDivisionError:
        return "penyebut tidak boleh nol"
```

## Mengapa tidak cukup menulis `except:` saja?

`except:` yang terlalu umum dapat menyembunyikan kesalahan logika yang seharusnya diperbaiki.

# Masalah 3: Newton untuk akar kuadrat

## Dari matematika

Untuk mencari $\sqrt{n}$, gunakan iterasi

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{n}{x_k}\right).$$

## Hitung manual dulu

Untuk $n = 9$ dan $x_0 = 4.5$, hitung $x_1, x_2, x_3$.

## Tahan dulu

Jangan buka kalkulator dulu. Kerjakan satu iterasi bersama.

# Masalah 3: Newton untuk akar kuadrat

## Dari matematika

Untuk mencari $\sqrt{n}$, gunakan iterasi

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{n}{x_k}\right).$$

## Hitung manual dulu

Untuk $n = 9$ dan $x_0 = 4.5$, hitung $x_1, x_2, x_3$.

$x_1 = \frac{1}{2}(4.5 + 9/4.5) = 3.25,$

$x_2 \approx 3.0096,$

$x_3 \approx 3.0000.$

# Kode Newton yang salah: apa masalahnya?

```
def akar(n, tol=1e-10):
    x = 0.0
    while abs(x*x - n) > tol:
        x = 0.5 * (x + n/x)
    return x
```

## Tebak dulu

1. Apa yang terjadi saat $n = 9$?
2. Apa yang terjadi saat $n = 0$?
3. Bagaimana jika $n < 0$?

# Kode Newton yang salah: apa masalahnya?

```
def akar(n, tol=1e-10):
    x = 0.0
    while abs(x*x - n) > tol:
        x = 0.5 * (x + n/x)
    return x
```

## Tebak dulu

1. Apa yang terjadi saat $n = 9$?
2. Apa yang terjadi saat $n = 0$?
3. Bagaimana jika $n < 0$?

## Masalah utama

Tebakan awal $x = 0$ membuat $n/x$ gagal. Kasus $n = 0$ dan $n < 0$ juga harus dipikirkan.

# Algoritma Newton yang lebih aman

```
def akar(n, tol=1e-10, max_iter=50):
    if n < 0:
        return None
    if n == 0:
        return 0.0

    x = n / 2 if n >= 1 else 1.0
    for k in range(max_iter):
        x_baru = 0.5 * (x + n/x)
        if abs(x_baru*x_baru - n) < tol:
            return x_baru
        x = x_baru
    return x
```

# Algoritma Newton yang lebih aman

```
def akar(n, tol=1e-10, max_iter=50):
    if n < 0:
        return None
    if n == 0:
        return 0.0

    x = n / 2 if n >= 1 else 1.0
    for k in range(max_iter):
        x_baru = 0.5 * (x + n/x)
        if abs(x_baru*x_baru - n) < tol:
            return x_baru
        x = x_baru
    return x
```

## Uji minimal

Coba $n = 0, 1, 4, 9, 2, -1$. Bandingkan dengan nilai yang sudah diketahui.

# Masalah 4: program jalan terus

```python
# A
i = n
while i > 0:
    i = i // 2
```

```python
# B
i = n
while i != 1:
    i = i - 2
```

```python
# C
i = n
while i > 0:
    i = i + 1
```

## Pertanyaan kelas

Untuk $n$ bilangan bulat positif, mana yang pasti berhenti? Mana yang hanya berhenti untuk sebagian input?

# Masalah 4: program jalan terus

```python
# A
i = n
while i > 0:
    i = i // 2
```

```python
# B
i = n
while i != 1:
    i = i - 2
```

```python
# C
i = n
while i > 0:
    i = i + 1
```

## Pertanyaan kelas

Untuk $n$ bilangan bulat positif, mana yang pasti berhenti? Mana yang hanya berhenti untuk sebagian input?

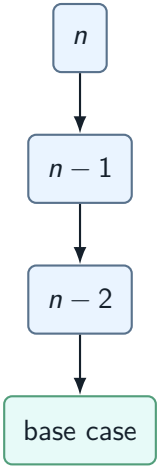
| Kode | Halt? | Alasan ringkas |
|---|---|---|
| A | Ya | $i$ turun: $n, n/2, n/4, \ldots, 0$ |
| B | Tergantung | berhenti untuk $n$ ganjil, tidak untuk $n$ genap |
| C | Tidak | $i$ bergerak menjauh dari 0 |

# Ukuran yang turun menuju berhenti

**Pertanyaan penting**

Pada loop atau rekursi, cari besaran yang makin dekat ke kondisi berhenti.

- Kode A: ukuran $i$ turun menjadi $\lfloor i/2 \rfloor$.
- Newton: galat $|x_k^2 - n|$ diharapkan mengecil.
- Rekursi faktorial: ukuran $n$ turun menjadi $n - 1$.



# Masalah 5: definisi faktorial

## Dari matematika

$$n! = n(n-1)!, \quad 0! = 1.$$

## Hitung manual

Tuliskan 5! tanpa kode.

## Tahan dulu

Biarkan mahasiswa menuliskan rantainya di papan.

# Masalah 5: definisi faktorial

## Dari matematika

$$n! = n(n-1)!, \quad 0! = 1.$$

## Hitung manual

Tuliskan 5! tanpa kode.

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120.$$

Pola ini cocok dengan pemanggilan fungsi pada masalah yang lebih kecil.

# Faktorial: rekursif dan iteratif

```python
def faktorial(n):
    if n == 0:
        return 1
    return n * faktorial(n-1)
```

```python
def faktorial_iter(n):
    hasil = 1
    for k in range(1, n+1):
        hasil *= k
    return hasil
```

# Faktorial: rekursif dan iteratif

```python
def faktorial(n):
    if n == 0:
        return 1
    return n * faktorial(n-1)
```

```python
def faktorial_iter(n):
    hasil = 1
    for k in range(1, n+1):
        hasil *= k
    return hasil
```

## Telusuri call stack

`faktorial(5)` menunggu hasil `faktorial(4)`, lalu `faktorial(3)`, dan seterusnya sampai base case.

# Masalah 6: barisan rekursif

**Diberikan**

$$a_0 = 2, \qquad a_n = 3a_{n-1} - 2, \quad n \geq 1.$$

**Kerjakan manual**

Hitung $a_0, a_1, a_2, a_3, a_4$. Tebak bentuk umum $a_n$.

**Tahan dulu**

Tulis tabel nilai bersama mahasiswa.

# Masalah 6: barisan rekursif

## Diberikan

$$a_0 = 2, \qquad a_n = 3a_{n-1} - 2, \quad n \geq 1.$$

## Kerjakan manual

Hitung $a_0, a_1, a_2, a_3, a_4$. Tebak bentuk umum $a_n$.

$$2,\ 4,\ 10,\ 28,\ 82, \ldots$$

Jika $b_n = a_n - 1$, maka $b_n = 3b_{n-1}$ dan $b_0 = 1$.

$$a_n = 3^n + 1.$$

# Barisan rekursif: implementasi dan pengecekan

```
def a(n):
    if n == 0:
        return 2
    return 3*a(n-1) - 2

for n in range(7):
    print(n, a(n), 3**n + 1)
```

# Barisan rekursif: implementasi dan pengecekan

```
def a(n):
    if n == 0:
        return 2
    return 3*a(n-1) - 2

for n in range(7):
    print(n, a(n), 3**n + 1)
```

## Yang diperiksa

- Base case: $n = 0$.
- Recursive case: $n$ bergerak ke $n - 1$.
- Verifikasi: hasil rekursif sama dengan rumus eksplisit untuk beberapa $n$.

# Masalah 7: submasalah dihitung berulang

## Diberikan

$$u_0 = 1, \quad u_1 = 3, \quad u_n = u_{n-1} + 2u_{n-2}.$$

## Pertanyaan

Gambar pohon pemanggilan untuk $u_5$. Submasalah mana yang muncul lebih dari sekali?

## Tahan dulu

Mulai dari $u_5 \to u_4$ dan $u_3$.

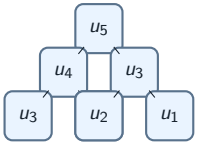
# Masalah 7: submasalah dihitung berulang

## Diberikan

$$u_0 = 1, \quad u_1 = 3, \quad u_n = u_{n-1} + 2u_{n-2}.$$

## Pertanyaan

Gambar pohon pemanggilan untuk $u_5$. Submasalah mana yang muncul lebih dari sekali?




## Catatan

Rekursi langsung mudah ditulis, tetapi dapat menghitung submasalah yang sama berkali-kali.

# Masalah 8: jarak Euclid dua vektor

## Dari matematika

Untuk $x, y \in \mathbb{R}^n$,

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}.$$

## Hitung manual

$d((0, 0), (3, 4)) = ?$

## Tahan dulu

Hubungkan dengan segitiga 3-4-5.

# Masalah 8: jarak Euclid dua vektor

## Dari matematika

Untuk $x, y \in \mathbb{R}^n$,

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}.$$

## Hitung manual

$d((0, 0), (3, 4)) = ?$

$$\sqrt{(3-0)^2 + (4-0)^2} = 5.$$

# Testing dengan assert

```
import math

def jarak(x, y):
    assert len(x) == len(y)
    s = 0
    for i in range(len(x)):
        s += (x[i] - y[i])**2
    return math.sqrt(s)

assert jarak([0,0], [0,0]) == 0
assert jarak([0,0], [3,4]) == 5
assert jarak([3,4], [0,0]) == jarak([0,0], [3,4])
```

# Testing dengan assert

```
import math

def jarak(x, y):
    assert len(x) == len(y)
    s = 0
    for i in range(len(x)):
        s += (x[i] - y[i])**2
    return math.sqrt(s)

assert jarak([0,0], [0,0]) == 0
assert jarak([0,0], [3,4]) == 5
assert jarak([3,4], [0,0]) == jarak([0,0], [3,4])
```

## Tambahkan sendiri

Kasus dimensi satu, komponen negatif, dan panjang vektor yang berbeda.

# Masalah 9: bilangan prima dan kondisi batas

```
def prima(n):
    if n <= 1:
        return False
    pembagi = 2
    while pembagi*pembagi < n:
        if n % pembagi == 0:
            return False
        pembagi += 1
    return True
```

## Prediksi

Uji secara mental untuk:

- $n = 2$
- $n = 9$
- $n = 49$

Mana yang membongkar bug?

# Masalah 9: bilangan prima dan kondisi batas

```
def prima(n):
    if n <= 1:
        return False
    pembagi = 2
    while pembagi*pembagi < n:
        if n % pembagi == 0:
            return False
        pembagi += 1
    return True
```

## Prediksi

Uji secara mental untuk:

- $n = 2$
- $n = 9$
- $n = 49$

Mana yang membongkar bug?

## Masalahnya

Syarat `< n` melewatkan pembagi saat `pembagi*pembagi == n`. Gunakan `<= n`.

# Baru sekarang kita beri nama istilahnya

| Istilah | Makna praktis di kelas ini |
|---|---|
| Syntax error | Bentuk kode tidak sah menurut aturan Python. |
| Runtime error | Kode sah, tetapi gagal saat dijalankan: pembagian nol, tipe salah, nama tidak ada. |
| Logical error | Program berjalan, tetapi hasilnya salah secara matematika atau algoritma. |
| Exception handling | Cara mengendalikan kegagalan yang dapat diperkirakan. |
| Assertion/test | Pemeriksaan kecil agar fungsi dipercaya pada kasus normal dan batas. |
| Terminasi | Program berhenti setelah langkah berhingga. |
| Base/recursive case | Bagian berhenti dan bagian pemanggilan ulang pada rekursi. |

# Langkah membaca kode bermasalah

1. Ambil contoh kecil yang bisa dihitung manual.
2. Catat nilai variabel setelah setiap baris penting.
3. Tanya: apakah loop atau rekursi mendekati kondisi berhenti?
4. Buat satu test yang seharusnya benar.
5. Buat satu test batas: nol, data kosong, bilangan negatif, panjang berbeda.
6. Perbaiki kode, lalu jalankan ulang test.

# Latihan kelas

## Kelompok kecil

Pilih satu dari masalah berikut, selesaikan di papan atau kertas, lalu jelaskan ke kelas.

1. Perbaiki fungsi rata-rata agar aman untuk data kosong.
2. Buat test untuk fungsi jarak Euclid.
3. Tentukan loop mana yang halt untuk $n = 6$ dan $n = 7$.
4. Telusuri call stack `faktorial(4)`.
5. Cari input yang membongkar bug fungsi `prima`.

# Quiz dan tugas mandiri M4

## Quiz e-learning

- Prediksi output kode salah.
- Klasifikasi masalah setelah contoh dibahas.
- Terminasi loop dan rekursi.
- Base case dan recursive case.

## Tugas mandiri

- Kerjakan soal M4 pada LKM.
- Sertakan perhitungan manual untuk minimal dua soal.
- Kode harus dapat dijalankan ulang dan memiliki test sederhana.

# Penutup

## Yang perlu dibawa pulang

Kode yang baik bukan hanya “berjalan”, tetapi juga dapat dijelaskan secara matematika, berhenti untuk input yang diizinkan, dan lolos uji kecil yang masuk akal.

